



OpenClaw: 从被动问答到主动执行的智能体演进



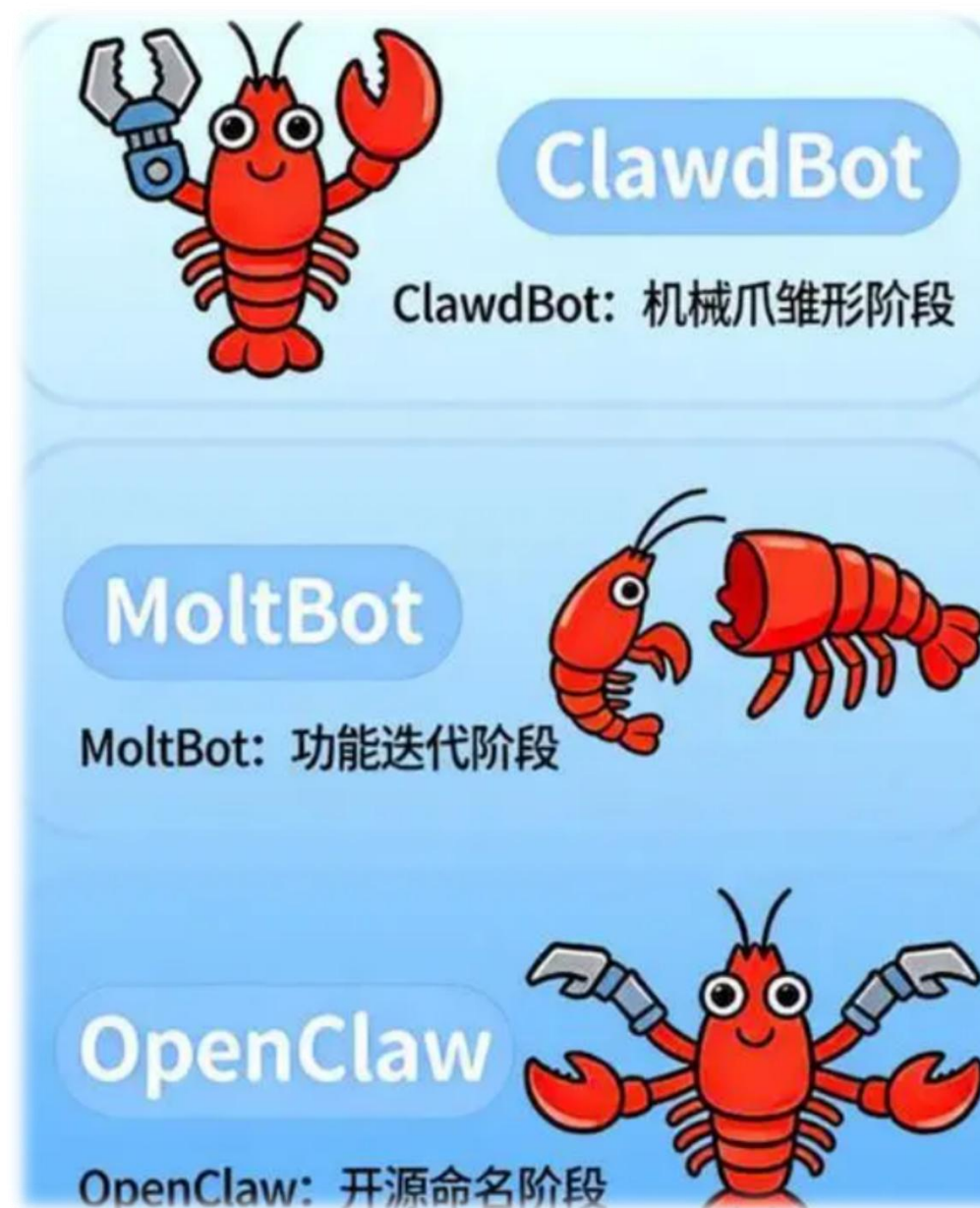
OpenClaw 的来历

OpenClaw是一款免费开源AI代理软件，由奥地利AI设计者**彼得·斯坦伯格 (Peter Steinberger)** 开发，去年11月推出。

OpenClaw最初并不叫这个名字。它最早叫**Clawdbot**，这个名字源自Anthropic公司的Claude大模型，初衷是为这个大模型加上能抓取任务的"手脚" (claw)。

2026年1月，Anthropic发邮件表示Clawdbot与Claude商标冲突，要求改名。开发团队在Discord连夜开会讨论，脑洞大开的名字层出不穷：Shelldon、Pinchy、小龙虾、穿背带的酒保龙虾等。

期间团队曾临时将名字定为**Moltbot** (molt是龙虾蜕壳的意思)，但这个名字念起来不顺口。最终在1月30号，项目正式更名为**OpenClaw**，既代表开源开放，又保留了"爪子"的核心形象。官方还打趣说"钳即是法"，将龙虾钳子和AI的执行力巧妙结合。





“龙虾” 的惊人引用量

GLOBAL TOP 10 GITHUB STAR RANKING

A Historic Milestone. March 2026.



No.1 OpenClaw

★ 245,119 Stars

Surpassing Linux in just 3 months.

No.2	 react	★ 243,411
No.3	 linux	★ 220,128
No.4	 Python	★ 218,250
No.5	 vue	★ 187,900
No.6	 tensorflow	★ 172,300
No.7	 freeCodeCamp	★ 320,000
No.8	 VSCode	★ 135,600
No.9	 flutter	★ 140,200
No.10	 oh-my-zsh	★ 128,500

截止到3月2日，OpenClaw登顶Github星标第一🔥，仅用3个月，那只超火的龙虾OpenClaw 🦞 就登顶了Github开源软件项目的星标**第一名**，超越了Linux、React等众多持续更新了十几年的超大型基建开源项目，目前已经高达325K

“百虾”大战

国内**13家科技大厂**和头部AI企业竞相跟进OpenClaw而引发的“百虾大战”，字节跳动、腾讯、阿里、京东、华为和百度等行业巨头纷纷依托自家云平台推出了一键部署服务，并将其深度融入飞书、企业微信、QQ及搜索框等核心应用中。

13家国内互联网大厂跟进OpenClaw

百虾大战

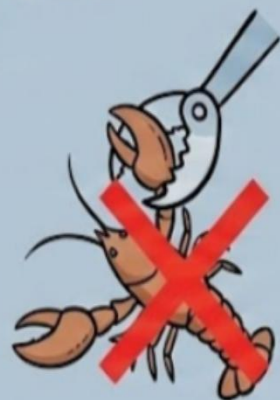
-  字节跳动 **1** ArkClaw + 飞书适配 + 火山引擎云部署
-  腾讯 **2** WorkBuddy + QClaw(内测) + 企微/QQ接入 + 腾讯云部署
-  京东 **3** 京东云一键部署
-  小米 **4** Xiaomi miclaw 手机系统层 Agent
-  华为 **5** 华为云一键部署  美团 联合联想百应远程部署服务
-  阿里 **7** 阿里云一键部署 + AgentBay
-  百度 **8** App搜索框接入 + 千帆 Skills + 百度能云部署
-  网易有道 **9** LobssterAI 桌面 Agent, 已开源
-  月之暗面 **10** Kimi Claw 托管版  KimClaw Claw 托管版
-  MiniMax **11** MaxClax 托管版 + 移动端
-  智谱 AI **12** AutoGLM OpenClaw 版
-  360 **13** 宣布将发布一键安装版(未落地)



“龙虾”到底有什么用？

【某平台发布的热门提问】

大家都在用
openclaw做
做什么呀



“你完全不需要
安装龙虾”



99%的人不需要安装龙虾

“openclaw就是
一坨能动的
垃圾”



很多人第一反应都是：“这玩意儿不就是个套壳的DeepSeek吗？和豆包有啥区别？”或者是看群里大家都在刷各种Skills(技能包)，自己也是一脑门子热全给安上了，结果发现这只“龙虾”还是傻乎乎的，问啥忘啥，根本没传说中那么神。



OpenClaw 到底是什么？

OpenClaw 是一款基于本地运行的跨平台全自动操作系统级**智能体**（**OS-Level Autonomous Agent**），无需人工逐步干预即可自主完成复杂任务链。



灵活交互

通过飞书、Slack 或 WeChat 等日常聊天软件直接下达任务指令。



能力扩展

依托 **ClawHub** 插件市场按需加载技能包，实现能力边界的无限延伸。



AI的演进之路：从大模型到智能体

阶段1：基础LLM

- 被动响应，无行动能力
- 仅基于训练数据生成文本
- 无法调用外部工具或API

阶段3：多步推理

- 具备简单规划能力
- 使用链式思维(CoT)等推理框架
- 但仍缺乏长期记忆和目标导向

阶段5：任务智能体

- 能理解目标、分解任务
- 执行多步行动并自我纠错
- 代表如AutoGPT、BabyAGI

阶段2：工具调用

- 能使用API，如搜索、计算器
- 具备初步的“手和脚”
- 仍是单轮、无状态的调用

阶段4：状态系统

- 拥有短期记忆和任务状态
- 能维护跨对话的上下文
- 属于“准智能体”的范畴

阶段6：自主协同--OpenClaw

- 高度自主，能主动行动
- 支持多智能体协作与学习
- 是AI向行动智能的终极跃迁

AI工具的进化之路：从「动嘴皮子」到「能干活」



📄 ChatGPT、DeepSeek、豆包：动嘴皮子

- 用户提问：'帮我看看怎么整理桌面文件'
- AI回复：提供详细的操作步骤和分类建议
- 用户行动：需要自己阅读、理解并手动执行所有步骤



🤖 OpenClaw：能干活

- 用户指令：'整理桌面文件，按类型分类，写总结明早发我邮箱'
- AI行动：自动分析文件、执行分类整理、生成总结报告
- 结果交付：用户第二天早上直接在邮箱中收到整理完毕的通知和总结文档

专属的“数字分身”



我们平时用的那些AI软件，本质上都是“路人甲”。你问它一句，它答你一句，聊完即走。但OpenClaw的底层逻辑完全不同。它更像是一个“数字生命”的胚胎。

OpenClaw: 你的“数字分身”

传统 AI ❌ 路人甲工具



OpenClaw

OpenClaw ✅



✅ 数字生命胚胎/数字分身

它不是“智能客服”，而是你的“系统”。
很多人觉得养龙虾难，其实只需要搞清楚三件事：

如何培养你的 OpenClaw 为“数字分身”？

- 1 它得有脑子 (文件系统)
- 2 它得有心跳 (Heartbeat)
- 3 它得有手脚 (Skills)



别让它像鱼一样只有七秒记忆。
喂关于你的信息，
建立只属于你的档案。



传统的 AI 是你不理它，
它就死在那；
OpenClaw 有心跳，在
你不说话时，自己去思考、
去主动找你。



技能包不是越多越好。
精准地给它配装备。

OpenClaw 的两大灵魂机制

核心策略：文件系统 + 记忆 Skills

为什么老是失忆？

你需要“文件系统”+“记忆 Skills”

很多朋友觉得 AI 像个失忆症患者，聊完就忘。那是因为普通的对话就像在沙滩上写字，潮水一冲就没了。

普通对话

无奈

1. 万物皆可文件 (Files)

在 OpenClaw 里，你要给它建一个专属的“知识库”。你的习惯、喜好、甚至你们聊过的深刻话题，都会被打包成一个个长期的“文件”存起来。

习惯
喜好
深刻话题

2. 安装“记忆 Skills”



光有文件还不行！要想让它真正懂你，你得给它安装专门的“记忆 Skills”。有了这个技能，AI 就相当于养成了一个好习惯：每次跟你说话前，都会先去翻翻你的文件库。

这样一来，它不仅彻底记住了你，还能完全贴合你的个人偏好来交流。

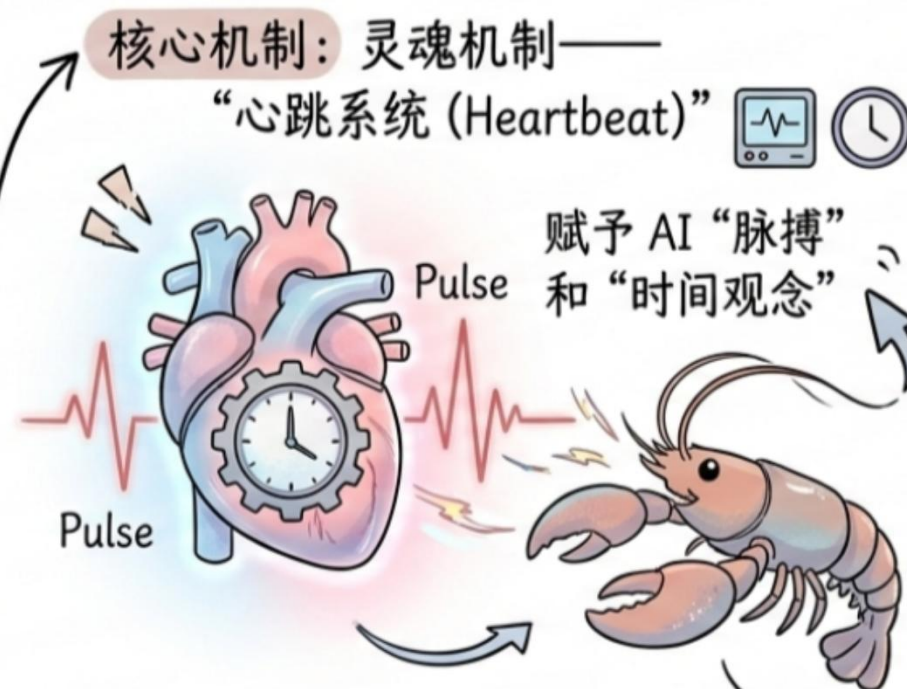


OpenClaw 的两大灵魂机制



核心策略：Heartbeat 机制

2. 怎么让它主动关心我？全靠灵魂机制：“心跳系统”



拥有生命感：从“被动等待”到
“主动出击”的转变（像活人）



Heartbeat 机制



OpenClaw按计划运行任务的两大机制

特性	OpenClaw Heartbeat	OpenClaw Cron
设计哲学	周期性"意识检查"	精确时间调度
运行环境	主会话 (Main Session)	主会话或独立会话
上下文	完整对话历史	可选隔离/共享
精度	大致间隔 (可漂移)	精确到秒
主要用途	批量监控检查	定时任务/提醒

Heartbeat:

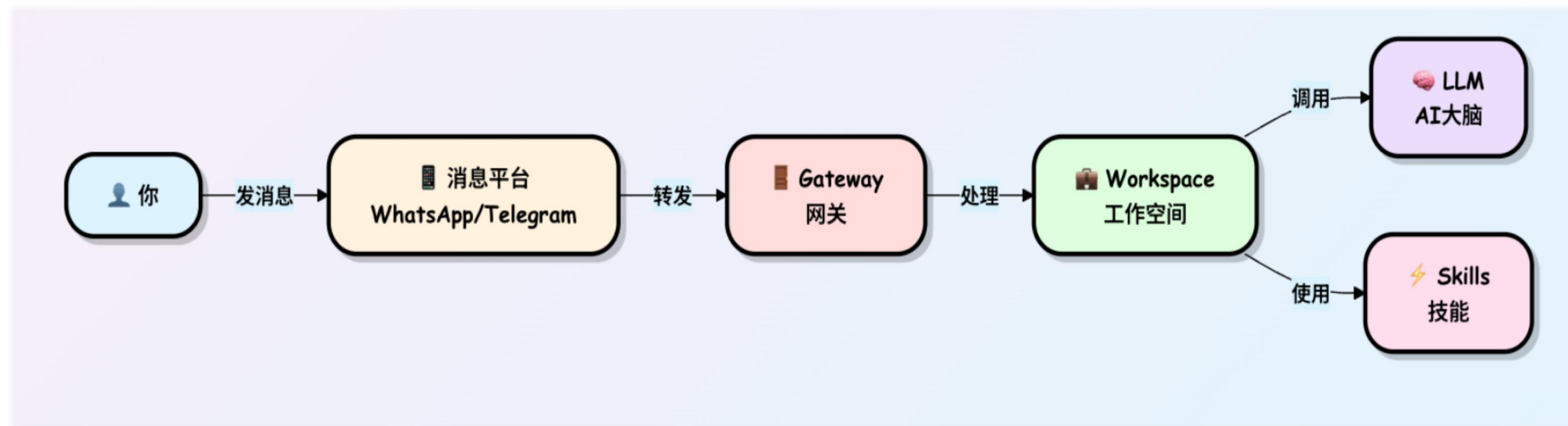
- 每 30 分钟一次性检查：收件箱、日历、通知、项目状态。
- 每30分钟总结主对话内容。

Cron:

- 与主对话无关的定时任务。
- 每天中午12点提醒我吃午饭。



OpenClaw 系统架构全景图



用户指令经由接口与消息渠道接入，通过**网关层鉴权**后送达工作空间，最终指挥工具执行器完成物理操作

OpenClaw 核心概念解析



OpenClaw 的架构由五个核心概念组成，每个都扮演着不同的角色：

概念	类比	作用
Gateway (网关)	大楼的前台接待	接收所有外部消息，分发到正确的工作空间
Workspace (工作空间)	你的私人办公室	处理具体任务，管理对话历史和技能
LLM (大语言模型)	AI顾问大脑	理解你的意图，生成回复
Skills (技能)	工具箱里的工具	执行特定功能（查天气、写代码、管理日程等）
Channels (渠道)	通信设备	连接不同的消息平台



Gateway:OpenClaw 的统一门户

消息接收、认证与路由的核心枢纽

Gateway（网关）是 OpenClaw 的大门，所有外部消息都必须先经过这里。它是唯一运行的进程，占用一个端口（默认 18789），像总机接线员一样把消息转发给 AI。

Gateway 的三大职责：

认证（Authentication）

确保只有你授权的平台才能连接，防止未授权的消息进入系统。

路由（Routing）

把消息送到正确的工作空间。如果你有多个工作空间（个人助理、工作助理等），Gateway 会智能分配。

日志记录（Logging）

记录所有交互，方便调试和审计，提供完整的操作追踪。

Gateway 的设计确保了 OpenClaw 的安全性和可扩展性，是整个系统的核心枢纽。



Workspace: 你的私人办公室

Workspace（工作空间）是实际处理任务的地方。你可以有多个工作空间，每个负责不同的事情





Workspace: 你的私人办公室

任务处理、对话管理与技能配置的核心场所；一个典型的 Workspace 配置示例：

```
# workspace-personal.yaml
name: "个人助理"
llm:
  provider: "anthropic" # 使用 Claude
  model: "claude-sonnet-4"
  apiKey: "sk-ant-xxx"

skills:
  - weather      # 查天气
  - calendar     # 管理日程
  - email        # 处理邮件
  - web-search   # 网页搜索

settings:
  language: "zh-CN"
  temperature: 0.7
  max_tokens: 4000
```

多工作空间使用场景：

- 1、个人助理工作空间 - 处理日常生活任务（查天气、管理日程、发邮件）
- 2、工作助理工作空间 - 处理工作相关任务（代码审查、文档生成、会议记录）
- 3、研究助理工作空间 - 处理学术任务（文献检索、数据分析、论文写作）



一条消息的完整旅程

让我们用一个真实例子来理解消息如何在 OpenClaw 中流转：场景是"在 WhatsApp 里说：帮我把今天的邮件整理成总结发给我"。



1、接收指令

你发消息 → 聊天软件的 Bridge 把消息推给 Gateway。



3、AI 思考

把消息 + 记忆打包发给 AI 大脑。AI 像聪明秘书："嗯，需要先读邮件，再总结。"



5、执行 + 反馈

工具把结果给 AI，AI 写出总结。



2、查找记忆

Gateway 打开你的"个人档案" (Session + Memory)，知道你是老用户、上次喜欢什么语气。



4、调用工具

AI 说"我要用 Gmail 技能"。Gateway 在安全沙箱里执行（防止AI乱改文件）。



6、回复用户

Gateway 把总结发回 WhatsApp。你就收到了！



Skills 系统：AI 的能力工具箱

技能 (Skills) 是 OpenClaw 执行特定任务的"能力模块"

每个技能包含:

- **定义 (Definition)** - 技能的名称、描述和参数
 - **权限 (Permissions)** - 该技能需要的系统权限
- **实现 (Implementation)** - 实际执行的代码逻辑
 - **错误处理 (Error Handling)** - 失败时的处理方式

OpenClaw 对技能有严格的安全控制:

权限	说明	示例
`network`	网络访问	查天气、搜索网页
`filesystem`	文件系统	读写文件
`email`	邮箱访问	发送/接收邮件
`calendar`	日历访问	管理日程
`system`	系统操作	执行命令

技能的调用流程:

1. AI 识别需要的技能

↓

2. 检查权限是否授予
3. 在沙箱环境中执行

↓

4. 返回结果给 AI
5. AI 生成最终回复

这种设计确保了 OpenClaw 既强大又安全。



OpenClaw Memory 完全指南

AI 的记忆困境

AI 在处理复杂任务和保持长期连贯性时，常面临以下瓶颈：

问题	表现	影响
短期记忆限制	上下文窗口有限（约 200k tokens）	长对话早期信息丢失
上下文噪声	无关信息淹没重要内容	响应质量下降
跨会话失忆	重启后从零开始	无法记住用户偏好

例如：用户上周曾强调“喜欢用简洁表格”，但 AI 在今日对话中因缺乏记忆，无法主动契合该偏好。

传统记忆文件结构

- MEMORY.md：长期记忆，存放稳定的重要信息（只在私聊中加载，群聊中不会暴露）
- memory/YYYY-MM-DD.md：每日笔记，记录当天的活动和上下文

局限性：单一 MEMORY.md 的局限性在于文件过大导致检索变慢、信息混杂且易产生更新冲突。



五层记忆架构

为解决 AI 记忆局限，OpenClaw 构建了五层分层记忆架构，实现高效的语义搜索与知识检索。

☐ 记忆体系：

L1 索引层 (MEMORY.md)

核心信息速查，作为各层的统一索引，记录关键决策与原则。

L2 项目层 (memory/projects.md)

跟踪项目状态、待办事项及相关决策记录。

L3 基础设施层 (memory/infra.md)

管理配置信息、API 密钥、服务器部署及定时任务。

L4 教训层 (memory/lessons.md)

分类记录致命错误、严重问题及经验教训，包含解决方案与预防措施。

L5 日志层 (memory/YYYY-MM-DD.md)

每日原始记录，遵循“先捕获，后提炼”的原则。

优势：

- L1层作为统一索引，快速定位。
- L2层追踪项目动态，提升了任务执行的精准度。
- L4层专门沉淀错误教训
- L5层遵循先捕获后提炼的原则
- 整体架构支持按需调用不同层级的信息，避免了单文件过大导致的干扰，比单纯的每日记录更适合管理长期项目



Skills 系统与进化逻辑

一、Skills 装备系统：为 AI 装上能力的"触角"

OpenClaw 并非全知全能，它更像是一位极其聪明但缺乏特定工具的"光杆司令"。
通过安装 **Skills (技能插件)**，你可以根据需求赋予它不同的专业能力。

能力定制化

你可以为它安装商业洞察 Skills 来辅助理财，或安装日程整理 Skills 来高效管家。

能力上限

Skills 直接决定了 AI 在现实场景中解决问题的能力边界。



Skills 系统与进化逻辑

二、自我进化：AI 的闭环反思机制

自我进化并非魔法，而是基于 **"进化 Skills"** 实现的持续反思与成长过程：

1 自动总结

任务完成后，AI 会在后台自动开启"总结会"进行复盘。

2 自我纠错

分析执行过程中的失误，总结更优思路，并确保下次改进。

3 写入长期记忆

将反思获得的经验再次写入文件库，实现能力的动态升级，越用越顺手。



OpenClaw vs. Claude Code: 架构差异的本质



两款AI Agent工具的根本区别不在能力，而在运行方式

1、OpenClaw

以**持久化后台**守护进程的方式运行，全天候驻留在你的机器上。

2、Claude Code

以**交互式会话**的形式启动，只在你主动使用时存在。

这一架构差异几乎解释了两款工具之间的所有其他差异——从记忆行为到安全态势，再到响应延迟。



OpenClaw: 本地守护进程的持久化运行

全天候驻留，持续监听，智能响应

OpenClaw 作为本地守护进程运行，大约占用 1GB 内存和 500MB 磁盘空间。启动后，它持续驻留在内存中，不断监听来自 **WhatsApp**、**Slack** 或 **Telegram** 等已连接平台的消息。

当你发送类似 "总结我未读的邮件，然后把下午3点的会议重新排期" 这样的消息时，OpenClaw 通过其 AI 引擎（可配置为使用 Claude、GPT-5、Kimi 2.5 甚至本地模型）处理请求，利用其包含 50 多个集成的工具库执行必要操作，并通过同一消息渠道回复。

持久化记忆

关键的是，OpenClaw 在对话间保持**持久化记忆**——它记住你的偏好、日历模式以及数周甚至数月前的对话上下文。最新稳定版 v2026.2.17 引入了改进的记忆压缩技术，在保持长期回忆准确性的同时降低存储需求。



Claude Code：交互式会话 的按需启动

专注编程，临时存在，高效响应

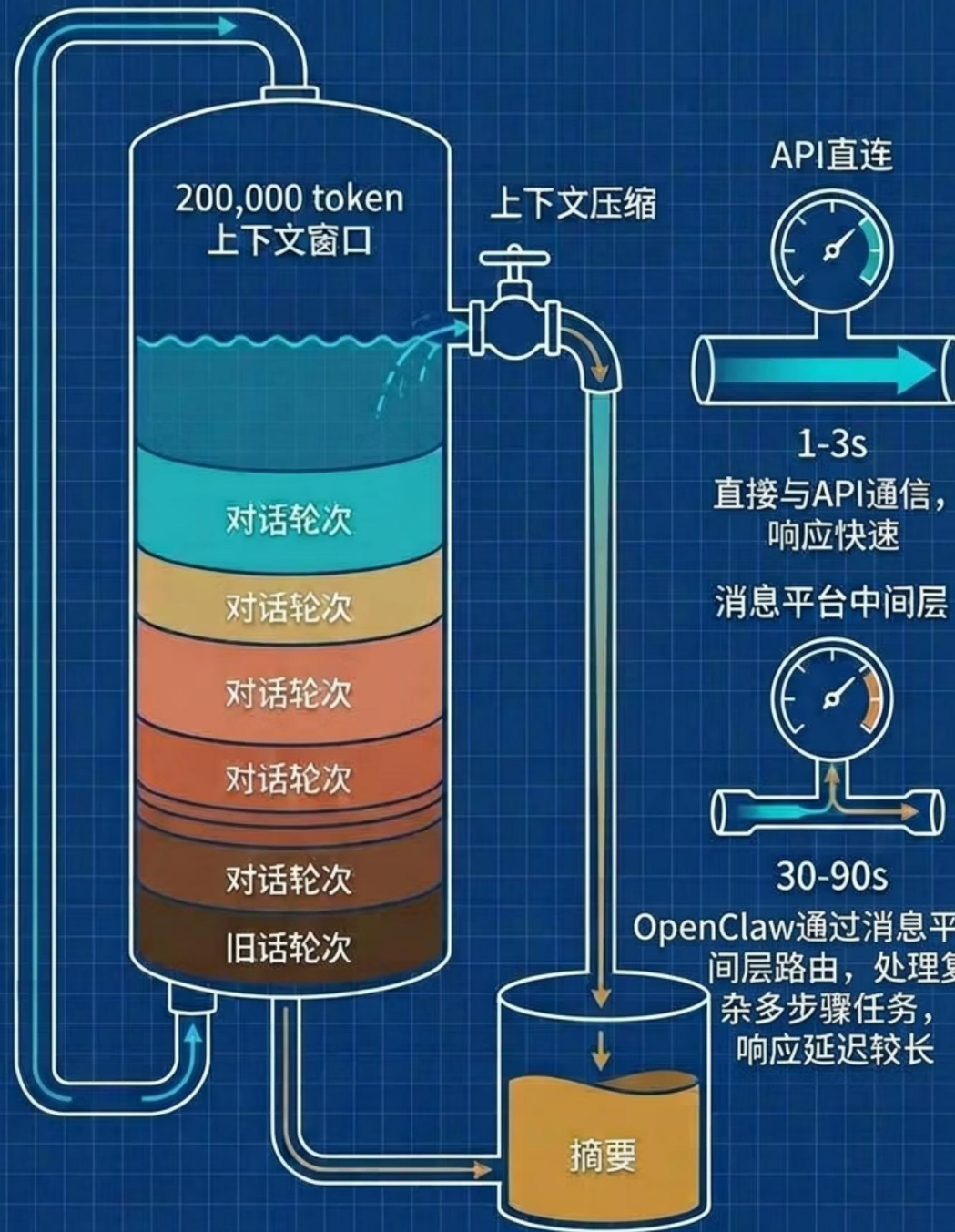
Claude Code 采取了截然不同的方式。当你在终端输入 `claude` 或通过 VS Code/JetBrains 打开会话时，一个新的智能体会话随即启动。该会话可以完全访问你当前项目中的文件系统、Shell命令和Git历史记录，但其记忆仅存在于会话期间——关闭终端后，上下文即消失。

Anthropic 通过上下文压缩技术来弥补这种临时性设计：当上下文窗口接近 200,000 token 限制时，智能地总结之前的对话轮次。同时通过 **CLAUDE.md 项目文件** 在会话间持久化指令。

会话架构的优势

会话架构意味着 **Claude Code** 高度专注——它不会被通知干扰，不能访问你的邮件，在你不编程时也不消耗资源。**响应延迟通常为 1-3 秒**，而 **OpenClaw 处理复杂多步骤任务需要 30-90 秒**，因为 Claude Code 直接与 Anthropic 的 API 通信，而非通过消息平台中间层路由。

Claude Code 架构详解 Session Architecture



CLAUDE.md 持久指令——跨会话保留



记忆在实践中如何运作

OpenClaw: 持久化记忆

使用 OpenClaw, 你可以说"记住我更喜欢用TypeScript而不是JavaScript开发新项目", 它会在未来**每次建议中考虑这个偏好**。你可以下个月问起上周的对话, 它能回忆起细节。这种持久化记忆使 OpenClaw 感觉更像一个了解你的私人助理——这正是它的设计目标。

Claude Code: 会话记忆

相比之下, Claude Code 将每次会话视为全新起点。它通过读取项目的 CLAUDE.md 文件、分析代码库结构和从现有代码推断约定来补偿——但它不会记得你昨天花了两小时调试一个竞态条件, 除非你**恢复同一会话或明确提供该上下文**。

这种差异决定了两款工具的适用场景: **OpenClaw 适合需要长期上下文和个人化的任务**, **Claude Code 适合需要深度专注和快速反馈的编程工作**。

API 路由机制与成本模型



模型成本策略对比

全用 Claude Sonnet (不优化)

主力模型: Claude Sonnet

输入价格 (每百万 Token) : \$3.00

相对成本: 100% (基准)

三级 Fallback (推荐)

主力模型: Sonnet → Haiku → DeepSeek

输入价格

Sonnet: \$3.00

Haiku: \$1.00

DeepSeek: \$0.14

相对成本: 约 5-20%

纯 DeepSeek (简单场景)

主力模型: DeepSeek-V3

输入价格 (每百万 Token) : \$0.14

相对成本: 约 5%



集成生态对比

OpenClaw

OpenClaw **内置50多个集成**，涵盖邮件（Gmail、Outlook）、日历（Google Calendar、Outlook Calendar）、消息（WhatsApp、Slack、Discord、Telegram）、任务管理（Todoist、Linear）、智能家居（HomeKit、Home Assistant）和浏览器（完整Web访问用于研究和自动化）。

每个集成都是功能完善的一级公民——例如Gmail集成不仅能发送通知，还能搜索、阅读、撰写、回复、标记和归档邮件。

Claude Code

Claude Code 的集成模型更加聚焦，但可通过模型上下文协议（MCP）进行**扩展**。开箱即用，Claude Code 与你的文件系统、Shell、Git和IDE（VS Code 和 JetBrains）集成。

MCP服务器将其扩展到数据库、文档服务、监控工具等。然而，设置MCP服务器需要比在 OpenClaw 配置文件中切换集成开关更高的技术能力。

安全与隐私：关键差异



OpenClaw：广泛访问的风险

OpenClaw 的安全模型授予守护进程访问你的邮件、日历、消息平台、Web浏览器、文件系统以及你启用的任何其他集成的权限。这种访问对于工具作为个人助手运作是必要的。然而，这种广泛的访问意味着 OpenClaw 一旦被入侵，理论上可能**暴露**你的邮件内容、日历事件、消息历史以及连接账户中存储的任何**凭证**。

Claude Code：沙箱隔离的保护

Claude Code 的能力来自对代码库的深度访问，但在严格**沙箱化的环境**内，限制了影响范围。它只在你主动使用时存在，并且其记忆仅存在于会话期间，极大地降低了潜在的攻击面和敏感信息泄露的风险。



漏洞危机

❑ **CVE-2026-25253**是OpenClaw历史上最严重的漏洞之一。安全研究人员给它起了一个代号："ClawJacked"。这个漏洞允许攻击者通过一个恶意链接实现"一键远程代码执行"——受害者只需要点击一个链接，攻击者就能完全控制他们的AI代理和系统

CVE-2026-25253 漏洞攻击链 (ClawJacked)

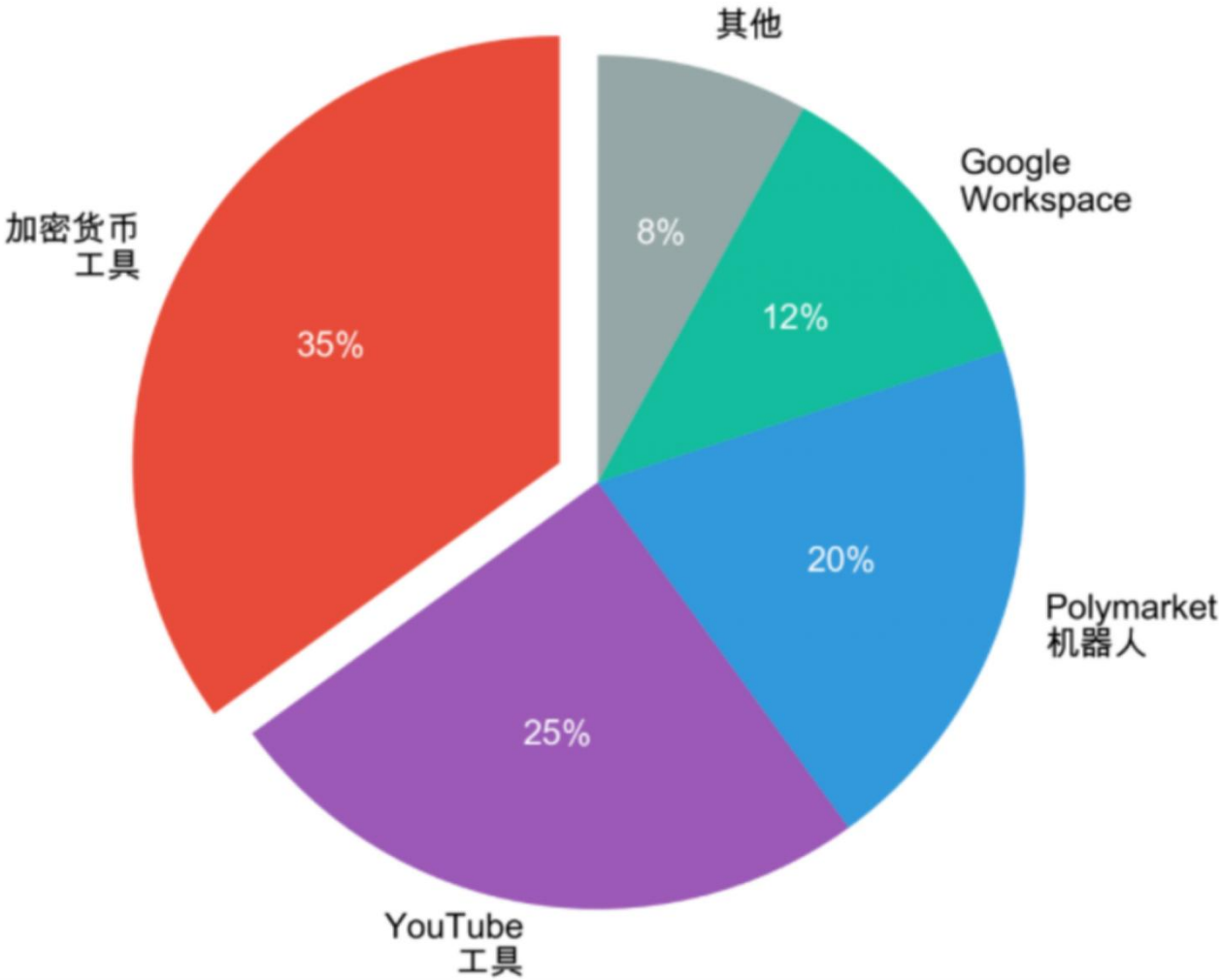
CVE-2026-25253 漏洞攻击链 (ClawJacked)



恶意技能



ClawHub 恶意技能伪装类型分布
(总计824个恶意技能)



ClawHub市场上恶意技能的伪装类型分布，其中**加密货币**相关工具占比最高，还包括YouTube工具、Polymarket**机器人**(15.870, -0.26, -1.61%)、Google Workspace集成等多种伪装形式。

OpenClaw**官方Skill仓库**中仍存在**恶意Skill**。该Skill伪装成"LinkedIn智能求职系统"，声称提供基于AI的职位搜索、简历定制、一键申请、面试跟踪及智能筛选等功能，具有高度迷惑性。



实践建议：如何安全使用

对于个人开发者来说，只要采取适当的预防措施，两款工具都可以安全使用。

对于 OpenClaw：

- 在VPN或防火墙后运行
- 保持更新到最新版本
- 在连接账户上启用双因素认证
- 定期审查其操作日志
- 避免在互联网上直接暴露实例

对于 Claude Code：

- 默认安全姿态已相当保守
- 主要风险是批准 Shell 命令时缺乏审查
- 始终在批准危险操作前仔细核对

针对企业团队的合规考量：

Claude Code 内置的控制和审计功能赋予了其明显优势。如果组织需要 **集中式策略管理**，Claude Code 是更好的选择。

总结：如果你需要全天候的个人助手和广泛的集成，**OpenClaw** 是理想选择；如果你需要深度的编程支持和企业级的安全控制，**Claude Code** 更合适。



具体使用场景：无处不在的代码助手

OpenClaw 将代码工作延伸至指尖。无论身在何处，通过日常聊天工具即可远程指挥复杂科研任务，实现高效无缝的工作体验。

轻量级交互

无需电脑，通过飞书、微信等应用发送指令。您的手机即是科研控制台。

远程智能调度

OpenClaw 自动协调云端工具，高效完成代码修改、部署及任务执行。

实时任务监控

随时随地掌握实验进展，异常情况即时推送到移动设备，确保科研不中断。

紧急问题处理

出游途中亦可远程修复代码 Bug，通过手机快速下达指令，保障项目进度。



结论



价值明确

在文献追踪、实验监控、任务编排三大场景具有显著效率优势。



风险可控

通过沙盒、审核、人在回路与本地化部署四层管控，将风险降至可控水平。



分步推进

从非敏感场景起步，稳步扩大应用边界，确保课题组利益始终优先。



Thanks